

VISVESVARAYA TECHNOLOGICAL UNIVERSITY

“JnanaSangama”, Belgaum -590014, Karnataka.



LAB REPORT
on

OPERATING SYSTEMS

Submitted by

ADITYA P MATHAPATI (1BM24CS021)

in partial fulfillment for the award of the degree of
BACHELOR OF ENGINEERING
in
COMPUTER SCIENCE AND ENGINEERING



B.M.S. COLLEGE OF ENGINEERING

(Autonomous Institution under VTU)

BENGALURU-560019

Feb-2026 to June-2026

B. M. S. College of Engineering,
Bull Temple Road, Bangalore 560019
(Affiliated To Visvesvaraya Technological University, Belgaum)
Department of Computer Science and Engineering



CERTIFICATE

This is to certify that the Lab work entitled “OPERATING SYSTEMS – 23CS4PCOPS” carried out by Aditya P Mathapati (1BM24CS021), who is Bonafide student of B. M. S. College of Engineering. It is in partial fulfilment for the award of Bachelor of Engineering in Computer Science and Engineering of the Visvesvaraya Technological University, Belgaum during the year Feb-2026 to June-2026. The Lab report has been approved as it satisfies the academic requirements in respect of a OPERATING SYSTEMS - (23CS4PCOPS) work prescribed for the said degree.

Dr. Seema Patil
Assistant Professor
Department of CSE
BMSCE, Bengaluru

Dr. Kavitha Sooda
Professor and Head
Department of CSE
BMSCE, Bengaluru

Index Sheet

Sl. No.	Experiment Title	Page No.
1.	Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time. →FCFS → SJF (pre-emptive & Non-preemptive)	1 - 4
2.	Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time. → Priority (pre-emptive & Non-pre-emptive) →Round Robin (Experiment with different quantum sizes for RR algorithm)	5 - 12
3.	Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.	13 – 16
4.	Write a C program to simulate Real-Time CPU Scheduling algorithms: a) Rate- Monotonic b) Earliest-deadline First c) Proportional scheduling	17 - 24
5.	Write a C program to simulate producer-consumer problem using semaphores	25 - 27
6.	Write a C program to simulate the concept of Dining Philosophers problem.	28 - 30
7.	Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.	31 - 36
8.	Write a C program to simulate the following contiguous memory allocation techniques. a) Worst-fit b) Best-fit c) First-fit	37 - 40
9.	Write a C program to simulate page replacement algorithms. a) FIFO b) LRU c) Optimal	41 - 45

Course Outcomes

C01	Apply the different concepts and functionalities of Operating System
C02	Analyse various Operating system strategies and techniques
C03	Demonstrate the different functionalities of Operating System.
C04	Conduct practical experiments to implement the functionalities of Operating system.

Program -1

Question:

Write a C program to simulate the following non-pre-emptive CPU scheduling algorithm to find turnaround time and waiting time.

→FCFS

→ SJF (pre-emptive & Non-preemptive)

Code:

=>FCFS:

```
#include <stdio.h>

struct Process {
    int ID, AT, BT, CT, TAT, WT, RT;
};

int main() {
    int n;
    printf("Enter no of processes : ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].ID = i + 1;
        printf("Arrival time of P%d : ", p[i].ID);
        scanf("%d", &p[i].AT);
        printf("Burst time of P%d : ", p[i].ID);
        scanf("%d", &p[i].BT);
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (p[j].AT > p[j + 1].AT) {
                struct Process temp = p[j];
                p[j] = p[j + 1];
                p[j + 1] = temp;
            }
        }
    }
    int timePassed = 0, SumTAT = 0, SumWT = 0, SumRT = 0;

    for (int i = 0; i < n; i++) {
```

```

    if (timePassed < p[i].AT) {
        timePassed = p[i].AT;
    }
    p[i].RT = timePassed - p[i].AT;
    p[i].CT = timePassed + p[i].BT;
    p[i].TAT = p[i].CT - p[i].AT;
    p[i].WT = p[i].TAT - p[i].BT;
    timePassed += p[i].BT;
    SumTAT += p[i].TAT;
    SumWT += p[i].WT;
    SumRT += p[i].RT;
}
printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\tRT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].ID, p[i].AT, p[i].BT, p[i].CT,
        p[i].TAT, p[i].WT, p[i].RT);
}
printf("\nAverage TAT : %.2f\n", (float)SumTAT / n);
printf("Average WT : %.2f\n", (float)SumWT / n);
printf("Average RT : %.2f\n", (float)SumRT / n);
return 0;
}

```

Result:

```

C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>FCFS
Enter no of processes : 4
Arrival time of P1 : 0
Burst time of P1 : 2
Arrival time of P2 : 1
Burst time of P2 : 2
Arrival time of P3 : 5
Burst time of P3 : 3
Arrival time of P4 : 6
Burst time of P4 : 4

Process AT      BT      CT      TAT      WT      RT
P1      0        2        2        2        0        0
P2      1        2        4        3        1        1
P3      5        3        8        3        0        0
P4      6        4       12        6        2        2

Average TAT : 3.50
Average WT : 0.75
Average RT : 0.75

```

=>SJF(Non-preemptive):

```
#include <stdio.h>

struct Process {
    int id, AT, BT, CT, TAT, WT, RT;
};

int main() {
    int n;
    printf("Enter no.of Processes: ");
    scanf("%d", &n);
    struct Process p[n];
    for (int i = 0; i < n; i++) {
        p[i].id = i + 1;
        printf("Arrival Time of P%d: ", p[i].id);
        scanf("%d", &p[i].AT);
        printf("Burst Time of P%d: ", p[i].id);
        scanf("%d", &p[i].BT);
    }
    int completed = 0, time = 0, SumTAT = 0, SumWT = 0;
    int isComplete[n];
    for (int i = 0; i < n; i++) isComplete[i] = 0;
    while (completed != n) {
        int idx = -1, minBT = 1e9;
        for (int i = 0; i < n; i++) {
            if (p[i].AT <= time && !isComplete[i] && p[i].BT < minBT) {
                minBT = p[i].BT;
                idx = i;
            }
        }
        if (idx != -1) {
            p[idx].CT = time + p[idx].BT;
            p[idx].TAT = p[idx].CT - p[idx].AT;
            p[idx].WT = p[idx].TAT - p[idx].BT;
            SumTAT += p[idx].TAT;
            SumWT += p[idx].WT;
            time += p[idx].BT;
            isComplete[idx] = 1;
            completed++;
        }
    }
}
```

```

        completed++;
    } else {
        time++;
    }
}

printf("\nProcess\tAT\tBT\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    printf("P%d\t%d\t%d\t%d\t%d\t%d\n", p[i].id, p[i].AT, p[i].BT, p[i].CT, p[i].TAT, p[i].WT);
}

printf("\nAverage TAT : %.2f\n", (float)SumTAT / n);
printf("Average WT : %.2f\n", (float)SumWT / n);
return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>gcc SJF_NPE.c -o SJF_NPE
```

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>SJF_NPE
```

```
Enter no.of Processes: 4
```

```
Arrival Time of P1: 0
```

```
Burst Time of P1: 6
```

```
Arrival Time of P2: 0
```

```
Burst Time of P2: 8
```

```
Arrival Time of P3: 0
```

```
Burst Time of P3: 7
```

```
Arrival Time of P4: 0
```

```
Burst Time of P4: 3
```

Process	AT	BT	CT	TAT	WT
P1	0	6	9	9	3
P2	0	8	24	24	16
P3	0	7	16	16	9
P4	0	3	3	3	0

```
Average TAT : 13.00
```

```
Average WT : 7.00
```


Program – 2

Question :

Write a C program to simulate the following CPU scheduling algorithm to find turnaround time and waiting time.

→ Priority (pre-emptive & Non-pre-emptive)

→ Round Robin (Experiment with different quantum sizes for RR algorithm)

Code :

⇒ Priority (pre-emptive)

```
#include <stdio.h>

struct Process {
    int pid, bt, at, priority, rt, wt, tat, ct;
};

int main() {
    int n;
    printf("Enter number of processes :");
    scanf("%d", &n);
    struct Process p[n];
    int isCompleted[n];
    for (int i = 0; i < n; i++) {
        p[i].pid = i + 1;
        printf("\nEnter Burst Time, Arrival Time and Priority for P%d:", p[i].pid);
        scanf("%d %d %d", &p[i].bt, &p[i].at, &p[i].priority);
        p[i].rt = p[i].bt;
        isCompleted[i] = 0;
    }
    int completed = 0, time = 0;
    while (completed != n) {
        int idx = -1;
        int highestPriority = 999999;
        for (int i = 0; i < n; i++) {
            if (p[i].at <= time && isCompleted[i] == 0) {
                if (p[i].priority < highestPriority) {
                    highestPriority = p[i].priority;
                    idx = i;
                } else if (p[i].priority == highestPriority) {
                    if (p[i].at < p[idx].at) {

```

```

        idx = i;
    }
}
}
}
if (idx != -1) {
    p[idx].rt--;
    time++;
    if (p[idx].rt == 0) {
        p[idx].ct = time;
        p[idx].tat = p[idx].ct - p[idx].at;
        p[idx].wt = p[idx].tat - p[idx].bt;
        isCompleted[idx] = 1;
        completed++;
    }
} else {
    time++;
}
}

float avgWT = 0, avgTAT = 0;
printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");
for (int i = 0; i < n; i++) {
    avgWT += p[i].wt;
    avgTAT += p[i].tat;
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].priority,
        p[i].ct, p[i].tat, p[i].wt);
}
printf("\nAverage TAT = %.2f", avgTAT / n);
printf("\nAverage WT = %.2f", avgWT / n);
return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)> Priority_PE
Enter number of processes :7

Enter Burst Time, Arrival Time and Priority for P1:5 6 6
Enter Burst Time, Arrival Time and Priority for P2:1 4 5
Enter Burst Time, Arrival Time and Priority for P3:2 1 4
Enter Burst Time, Arrival Time and Priority for P4:4 3 4
Enter Burst Time, Arrival Time and Priority for P5:8 0 3
Enter Burst Time, Arrival Time and Priority for P6:6 5 2
Enter Burst Time, Arrival Time and Priority for P7:1 10 1

Process AT      BT      Priority      CT      TAT      WT
P1      6      5      6      27      21      16
P2      4      1      5      22      18      17
P3      1      2      4      17      16      14
P4      3      4      4      21      18      14
P5      0      8      3      15      15      7
P6      5      6      2      12      7      1
P7     10      1      1      11      1      0

Average TAT = 13.71
Average WT = 9.86
```

⇒ Priority(Non-pre-emptive)

```
#include<stdio.h>

struct Process{
int pid,bt,at,priority,wt,tat,ct;};

int main(){
int n;
printf("Enter number of processes :");
scanf("%d",&n);
struct Process p[n];
for (int i=0;i<n;i++){
    p[i].pid=i+1;
    printf("\nEnter Burst Time, Arrival Time and Priority for P%d:",p[i].pid);
    scanf("%d %d %d",&p[i].bt,&p[i].at,&p[i].priority);
}
```

```

int completed=0,time=0;
int isCompleted[n];
for(int i=0;i<n;i++){
    isCompleted[i]=0;
}
while(completed!=n){
    int idx=-1;
    int highestPriority=999999;
    for(int i=0;i<n;i++){
        if(p[i].at<=time && isCompleted[i]==0){
            if(p[i].priority<highestPriority){
                highestPriority=p[i].priority;
                idx=i;
            }
            else if(p[i].priority==highestPriority){
                if(p[i].at<p[idx].at){
                    idx=i;
                }
            }
        }
    }
    if(idx!=-1){
        p[idx].ct=time+p[idx].bt;
        p[idx].tat=p[idx].ct-p[idx].at;
        p[idx].wt=p[idx].tat-p[idx].bt;
        time=p[idx].ct;
        isCompleted[idx]=1;
        completed++;
    }
    else{
        time++;
    }
}
float avgWT=0, avgTAT=0;
printf("\nProcess\tAT\tBT\tPriority\tCT\tTAT\tWT\n");
for(int i=0;i<n;i++){

```

```

    avgWT+=p[i].wt;
    avgTAT+=p[i].tat;
    printf("P%d\t%d\t%d\t%d\t\t%d\t%d\t%d\n",p[i].pid,p[i].at,p[i].bt,p[i].priority,p[i].ct,p[i].tat,p[i].wt);
}
printf("\nAverage TAT = %.2f",avgTAT/n);
printf("\nAverage WT = %.2f",avgWT/n);
return 0;
}

```

Result :

```

C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>gcc Priority_NPE.c -o Priority_NPE
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Priority_NPE
Enter number of processes :5

Enter Burst Time, Arrival Time and Priority for P1:3 0 5
Enter Burst Time, Arrival Time and Priority for P2:2 2 3
Enter Burst Time, Arrival Time and Priority for P3:5 3 2
Enter Burst Time, Arrival Time and Priority for P4:4 4 4
Enter Burst Time, Arrival Time and Priority for P5:1 6 1

Process AT      BT      Priority      CT      TAT      WT
P1      0        3        5          3        3        0
P2      2        2        3         11        9        7
P3      3        5        2          8        5        0
P4      4        4        4         15       11        7
P5      6        1        1          9        3        2

Average TAT = 6.20
Average WT = 3.20

```

⇒ Round Robin

```

#include <stdio.h>

struct Process {
    int pid, at, bt, ct, tat, wt, rt;
};

int main() {
    int n, tq;
    printf("Enter number of processes: ");
    scanf("%d", &n);
    printf("Enter Time Quantum: ");
    scanf("%d", &tq);
}

```

```

struct Process p[n];
int rem_bt[n];
int readyQ[100];
int front = 0, rear = 0;
printf("Enter arrival times:\n");
for(int i=0; i<n; i++) {
    scanf("%d", &p[i].at);
    p[i].pid = i+1;
}
printf("Enter burst times:\n");
for(int i=0; i<n; i++) {
    scanf("%d", &p[i].bt);
    rem_bt[i] = p[i].bt;
    p[i].rt = p[i].bt;
    p[i].ct = p[i].tat = p[i].wt = 0;
}
int time = 0, completed = 0;
int visited[n];
for(int i=0; i<n; i++) visited[i] = 0;
for(int i=0; i<n; i++) {
    if(p[i].at == 0) {
        readyQ[rear++] = i;
        visited[i] = 1;
    }
}
while(completed < n) {
    if(front == rear) {
        time++;
        for(int i=0; i<n; i++) {
            if(!visited[i] && p[i].at <= time) {
                readyQ[rear++] = i;
                visited[i] = 1;
            }
        }
        continue;
    }
}

```

```

int idx = readyQ[front++];
if(rem_bt[idx] > tq) {
    time += tq;
    rem_bt[idx] -= tq;
    p[idx].rt = rem_bt[idx];
} else {
    time += rem_bt[idx];
    rem_bt[idx] = 0;
    p[idx].ct = time;
    p[idx].tat = p[idx].ct - p[idx].at;
    p[idx].wt = p[idx].tat - p[idx].bt;
    p[idx].rt = 0;
    completed++;
}
for(int i=0; i<n; i++) {
    if(!visited[i] && p[i].at <= time) {
        readyQ[rear++] = i;
        visited[i] = 1;
    }
}
if(rem_bt[idx] > 0) {
    readyQ[rear++] = idx;
}
}
printf("\nRRS scheduling:\n");
printf("PID\tAT\tBT\tCT\tTAT\tWT\n");
for(int i=0; i<n; i++) {
    printf("%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt, p[i].ct,
        p[i].tat, p[i].wt, p[i].rt);
}
float avg_TAT = 0, avg_WT = 0;
for(int i=0; i<n; i++) {
    avg_TAT += p[i].tat;
    avg_WT += p[i].wt;
}

```

```

printf("\nAverage turnaround time: %.2f ms\n", avg_TAT/n);
printf("Average waiting time: %.2f ms\n", avg_WT/n);
return 0;
}

```

Result :

```

C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>gcc RoundRobin.c -o RoundRobin
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>RoundRobin
Enter number of processes: 5
Enter Time Quantum: 2
Enter arrival times:
0 1 2 3 4
Enter burst times:
5 3 1 2 3

RRS scheduling:

```

PID	AT	BT	CT	TAT	WT
1	0	5	13	13	8
2	1	3	12	11	8
3	2	1	5	3	2
4	3	2	9	6	4
5	4	3	14	10	7

```

Average turnaround time: 8.60 ms
Average waiting time: 5.80 ms

```


Program – 3

Question :

Write a C program to simulate multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code :

```
#include <stdio.h>

#define MAX 10

typedef struct {
    int pid, at, bt, rt, ct, tat, wt, queue;
} Process;

int main() {
    Process p[MAX];
    int n, i, time = 0, completed = 0;
    int tq = 2;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    for (i = 0; i < n; i++) {
        printf("\nP%d\n", i + 1);
        p[i].pid = i + 1;

        printf("Arrival Time: ");
        scanf("%d", &p[i].at);

        printf("Burst Time: ");
        scanf("%d", &p[i].bt);

        printf("Queue (1-System, 2-User): ");
        scanf("%d", &p[i].queue);
    }
}
```

```

        p[i].rt = p[i].bt;
    }

    printf("\nGantt Chart:\n");

    while (completed < n) {

        int executed = 0;

        for (i = 0; i < n; i++) {
            if (p[i].queue == 1 && p[i].rt > 0 && p[i].at <= time) {

                executed = 1;

                if (p[i].rt > tq) {
                    printf("P%d ", p[i].pid);
                    time += tq;
                    p[i].rt -= tq;
                } else {
                    printf("P%d ", p[i].pid);
                    time += p[i].rt;
                    p[i].rt = 0;
                    p[i].ct = time;
                    completed++;
                }
            }
        }

        if (!executed) {
            for (i = 0; i < n; i++) {
                if (p[i].queue == 2 && p[i].rt > 0 && p[i].at <= time) {

                    executed = 1;

                    printf("P%d ", p[i].pid);
                    p[i].rt--;
                }
            }
        }
    }
}

```

```

        time++;

        if (p[i].rt == 0) {
            p[i].ct = time;
            completed++;
        }

        break;
    }
}

if (!executed) {
    time++;
}
}

```

```

float total_tat = 0, total_wt = 0;

```

```

printf("\n\nPID\tAT\tBT\tCT\tTAT\tWT\n");

```

```

for (i = 0; i < n; i++) {
    p[i].tat = p[i].ct - p[i].at;
    p[i].wt = p[i].tat - p[i].bt;

    total_tat += p[i].tat;
    total_wt += p[i].wt;

    printf("P%d\t%d\t%d\t%d\t%d\t%d\n",
        p[i].pid, p[i].at, p[i].bt,
        p[i].ct, p[i].tat, p[i].wt);
}

```

```

printf("\nAverage TAT = %.2f", total_tat / n);
printf("\nAverage WT = %.2f\n", total_wt / n);

```

```
return 0;
}
```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>gcc MultilevelQueue.c -o MultilevelQueue
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>MultilevelQueue
Enter number of processes: 4

P1
Arrival Time: 0
Burst Time: 4
Queue (1-System, 2-User): 1

P2
Arrival Time: 0
Burst Time: 3
Queue (1-System, 2-User): 1

P3
Arrival Time: 0
Burst Time: 8
Queue (1-System, 2-User): 2

P4
Arrival Time: 10
Burst Time: 5
Queue (1-System, 2-User): 1

Gantt Chart:
P1 P2 P1 P2 P3 P3 P3 P4 P4 P4 P3 P3 P3 P3 P3

PID    AT    BT    CT    TAT    WT
P1      0     4     6     6     2
P2      0     3     7     7     4
P3      0     8    20    20    12
P4     10     5    15     5     0

Average TAT = 9.50
Average WT = 4.50
```

Program – 4

Question :

Write a C program to simulate Real-Time CPU Scheduling algorithms:

- a) Rate- Monotonic
- b) Earliest-deadline First
- c) Proportional scheduling

Code :

⇒ Rate- Monotonic

```
#include <stdio.h>
#include<math.h>
#define MAX 10
int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}
int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}
int main() {
    int n;
    int burst[MAX], period[MAX];
    int hyperperiod;
    printf("Enter the number of processes: ");
    scanf("%d", &n);
    printf("Enter the CPU burst times:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &burst[i]);
    }
    printf("Enter the time periods:\n");
    for (int i = 0; i < n; i++) {
        scanf("%d", &period[i]);
    }
    hyperperiod = period[0];
    for (int i = 1; i < n; i++) {
        hyperperiod = lcm(hyperperiod, period[i]);
    }
    printf("LCM=%d\n\n", hyperperiod);
    printf("Rate Monotone Scheduling:\n");
    printf("PID  Burst  Period\n");
    for (int i = 0; i < n; i++) {
        printf("%d    %d    %d\n", i + 1, burst[i], period[i]);
    }
}
```

```

double utilization = 0.0;
for (int i = 0; i < n; i++) {
    utilization += (double)burst[i] / period[i];
}
double bound = n * (pow(2.0, 1.0 / n) - 1);
printf("\n%f <= %f => %s\n", utilization, bound,
    (utilization <= bound) ? "true" : "false");
printf("Scheduling occurs for %d ms\n\n", hyperperiod);
int remaining[MAX] = {0};
for (int time = 0; time < hyperperiod; time++) {
    for (int i = 0; i < n; i++) {
        if (time % period[i] == 0) {
            remaining[i] = burst[i];
        }
    }
    int selected = -1;
    for (int i = 0; i < n; i++) {
        if (remaining[i] > 0) {
            selected = i;
            break;
        }
    }
    if (selected != -1) {
        if (remaining[selected] == burst[selected]) {
            printf("%dms onwards: Process %d running\n", time, selected + 1);
        }
        remaining[selected]--;
    } else {
        printf("%dms onwards: CPU is idle\n", time);
    }
}
return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>RMS
Enter the number of processes: 2
Enter the CPU burst times:
20 35
Enter the time periods:
50 100
LCM=100

Rate Monotone Scheduling:
PID    Burst    Period
1      20       50
2      35      100

0.750000 <= 0.828427 => true
Scheduling occurs for 100 ms

0ms onwards: Process 1 running
20ms onwards: Process 2 running
50ms onwards: Process 1 running
75ms onwards: CPU is idle
76ms onwards: CPU is idle
77ms onwards: CPU is idle
78ms onwards: CPU is idle
79ms onwards: CPU is idle
80ms onwards: CPU is idle
81ms onwards: CPU is idle
82ms onwards: CPU is idle
83ms onwards: CPU is idle
84ms onwards: CPU is idle
85ms onwards: CPU is idle
86ms onwards: CPU is idle
87ms onwards: CPU is idle
88ms onwards: CPU is idle
89ms onwards: CPU is idle
90ms onwards: CPU is idle
91ms onwards: CPU is idle
92ms onwards: CPU is idle
93ms onwards: CPU is idle
94ms onwards: CPU is idle
95ms onwards: CPU is idle
96ms onwards: CPU is idle
97ms onwards: CPU is idle
98ms onwards: CPU is idle
99ms onwards: CPU is idle
```

⇒ Earliest-deadline First

```
#include <stdio.h>
#define MAX_TASKS 10
typedef struct {
    int id;
    int execution_time;
    int period;
    int relative_deadline;
    int remaining_time;
    int deadline;
} Task;

int gcd(int a, int b) {
    while (b != 0) {
        int temp = b;
        b = a % b;
        a = temp;
    }
    return a;
}

int lcm(int a, int b) {
    return (a * b) / gcd(a, b);
}

int main() {
    int n, hyperperiod;
    printf("Enter number of tasks: ");
    scanf("%d", &n);
    Task tasks[MAX_TASKS];
    printf("\nEnter Burst Time, Period, and Deadline for each task:\n");
    for (int i = 0; i < n; i++) {
        tasks[i].id = i + 1;
        printf("\nTask %d Burst Time: ", i + 1);
        scanf("%d", &tasks[i].execution_time);
        printf("Task %d Period: ", i + 1);
        scanf("%d", &tasks[i].period);
        printf("Task %d Deadline: ", i + 1);
        scanf("%d", &tasks[i].relative_deadline);
        tasks[i].remaining_time = 0;
        tasks[i].deadline = tasks[i].relative_deadline;
    }
    hyperperiod = tasks[0].period;
    for (int i = 1; i < n; i++) {
```



```

    hyperperiod = lcm(hyperperiod, tasks[i].period);
}
printf("\nPID\tBurst\tDeadline\tPeriod\n");
for (int i = 0; i < n; i++) {
    printf("%d\t%d\t%d\t%d\n",
        tasks[i].id,
        tasks[i].execution_time,
        tasks[i].relative_deadline,
        tasks[i].period);
}
printf("\nScheduling occurs for %d ms\n\n", hyperperiod);
for (int time = 0; time < hyperperiod; time++) {
    for (int i = 0; i < n; i++) {
        if (time % tasks[i].period == 0) {
            tasks[i].remaining_time = tasks[i].execution_time;
            tasks[i].deadline = time + tasks[i].relative_deadline;
        }
    }
    int selected = -1;
    int earliest_deadline = 100000;
    for (int i = 0; i < n; i++) {
        if (tasks[i].remaining_time > 0 &&
            tasks[i].deadline < earliest_deadline) {
            earliest_deadline = tasks[i].deadline;
            selected = i;
        }
    }
    if (selected != -1) {
        printf("%dms : Task %d is running.\n", time, tasks[selected].id);
        tasks[selected].remaining_time--;
    } else {
        printf("%dms : CPU is idle.\n", time);
    }
}
return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>EDF
Enter number of tasks: 3
```

```
Enter Burst Time, Period, and Deadline for each task:
```

```
Task 1 Burst Time: 2
Task 1 Period: 5
Task 1 Deadline: 4
```

```
Task 2 Burst Time: 3
Task 2 Period: 20
Task 2 Deadline: 7
```

```
Task 3 Burst Time: 2
Task 3 Period: 10
Task 3 Deadline: 8
```

PID	Burst	Deadline	Period
1	2	4	5
2	3	7	20
3	2	8	10

```
Scheduling occurs for 20 ms
```

```
0ms : Task 1 is running.
1ms : Task 1 is running.
2ms : Task 2 is running.
3ms : Task 2 is running.
4ms : Task 2 is running.
5ms : Task 3 is running.
6ms : Task 3 is running.
7ms : Task 1 is running.
8ms : Task 1 is running.
9ms : CPU is idle.
10ms : Task 1 is running.
11ms : Task 1 is running.
12ms : Task 3 is running.
13ms : Task 3 is running.
14ms : CPU is idle.
15ms : Task 1 is running.
16ms : Task 1 is running.
17ms : CPU is idle.
18ms : CPU is idle.
19ms : CPU is idle.
```

⇒ Proportional scheduling

```
#include <stdio.h>

int main()
{
    int n, i, total = 0, timePeriod;
    int tickets[10];
    int picked[] = {29, 55, 47, 30, 12};
    printf("Enter number of processes: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++)
    {
        printf("\nProcess %d:\n", i + 1);
        printf("Tickets: ");
        scanf("%d", &tickets[i]);
        total += tickets[i];
    }
    printf("\n--- Proportional Share Scheduling ---\n");
    printf("\nEnter the Time period for scheduling: ");
    scanf("%d", &timePeriod);
    for(i = 0; i < timePeriod; i++)
    {
        int winner;
        if(picked[i] <= tickets[0])
            winner = 1;
        else if(picked[i] <= tickets[0] + tickets[1])
            winner = 2;
        else
            winner = 3;
        printf("\nTickets picked : %d, Winner : P%d",
            picked[i], winner);
    }

    printf("\n\n");
    for(i = 0; i < n; i++)
    {
        float percent = ((float)tickets[i] / total) * 100;
```

```

    printf("The Process : P%d gets %.2f%% of Processor Time.\n",
        i + 1, percent);
}
return 0;
}

```

Result :

```

C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Proportional
Enter number of processes: 3

Process 1:
Tickets: 10

Process 2:
Tickets: 20

Process 3:
Tickets: 30

--- Proportional Share Scheduling ---

Enter the Time period for scheduling: 5

Tickets picked : 29, Winner : P2
Tickets picked : 55, Winner : P3
Tickets picked : 47, Winner : P3
Tickets picked : 30, Winner : P2
Tickets picked : 12, Winner : P2

The Process : P1 gets 16.67% of Processor Time.
The Process : P2 gets 33.33% of Processor Time.
The Process : P3 gets 50.00% of Processor Time.

```

Program – 5

Question :

Write a C program to simulate producer-consumer problem using semaphores

Code :

```
#include <stdio.h>
#include <stdlib.h>

#define BUFFER_SIZE 5

int buffer[BUFFER_SIZE];
int in = 0, out = 0;
int mutex = 1;
int full = 0;
int empty = BUFFER_SIZE;
void wait(int *s) {
    (*s)--;
}

void signal(int *s) {
    (*s)++;
}

void producer(int next_prod) {
    //Overflow
    if (empty != 0 && mutex == 1) {
        wait(&empty);
        wait(&mutex);

        buffer[in] = next_prod;
        printf("Producer produced: %d at position %d\n", next_prod, in);
        in = (in + 1) % BUFFER_SIZE;

        signal(&mutex);
        signal(&full);
    } else {
        printf("Buffer full, producer waiting...\n");
    }
}

void consumer() {
    int next_cons;
    if (full != 0 && mutex == 1) {
        wait(&full);
        wait(&mutex);

        next_cons = buffer[out];
        printf("Consumer consumed: %d from position %d\n", next_cons, out);
        out = (out + 1) % BUFFER_SIZE;
```

```

        signal(&mutex);
        signal(&empty);
    } else {
        printf("Buffer empty, consumer waiting...\n");
    }
}

int main() {
    int choice, item;

    while (1) {
        printf("\n1. Produce\n2. Consume\n3. Exit\nEnter choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter item to produce: ");
                scanf("%d", &item);
                producer(item);
                break;
            case 2:
                consumer();
                break;
            case 3:
                printf("USN:1BM24CS021");
                exit(0);
            default:
                printf("Invalid choice!\n");
        }
    }
    return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Bounded_Buffer
```

```
1. Produce
```

```
2. Consume
```

```
3. Exit
```

```
Enter choice: 1
```

```
Enter item to produce: 10
```

```
Producer produced: 10 at position 0
```

```
1. Produce
```

```
2. Consume
```

```
3. Exit
```

```
Enter choice: 1
```

```
Enter item to produce: 20
```

```
Producer produced: 20 at position 1
```

```
1. Produce
```

```
2. Consume
```

```
3. Exit
```

```
Enter choice: 1
```

```
Enter item to produce: 30
```

```
Producer produced: 30 at position 2
```

```
1. Produce
```

```
2. Consume
```

```
3. Exit
```

```
Enter choice: 2
```

```
Consumer consumed: 10 from position 0
```

```
1. Produce
```

```
2. Consume
```

```
3. Exit
```

```
Enter choice: 3
```

```
USN:1BM24CS021
```

Program – 6

Question :

Write a C program to simulate the concept of Dining Philosophers problem.

Code :

```
#include<stdio.h>
#include<pthread.h>
#include<unistd.h>
#define N 5
pthread_mutex_t forks[N];
pthread_t philosophers[N];
void* philosopher(void* num)
{
    int id = *(int*)num;
    int left = id;
    int right = (id + 1) % N;
    while (1)
    {
        printf("Philosopher %d is thinking.\n", id);
        sleep(1);
        if (id % 2 == 0)
        {
            pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);

            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right); }
        else
        {
            pthread_mutex_lock(&forks[right]);
            printf("Philosopher %d picked up right fork %d.\n", id, right);
            fork pthread_mutex_lock(&forks[left]);
            printf("Philosopher %d picked up left fork %d.\n", id, left);
        }
        printf("Philosopher %d is eating.\n", id);
        sleep(2);
        pthread_mutex_unlock(&forks[left]);
        pthread_mutex_unlock(&forks[right]);
        printf("Philosopher %d put down forks %d and %d.\n", id, left, right);
    }
    return NULL;
}

int main() {
    int i;
    int ids[N];
    for (i = 0; i < N; i++)
    { pthread_mutex_init(&forks[i], NULL);
      ids[i] = i;
```



```
}  
for (i = 0; i < N; i++)  
  
{  
pthread_create(&philosophers[i], NULL, philosopher, &ids[i]);  
}  
for (i = 0; i < N; i++)  
{  
pthread_join(philosophers[i], NULL); }  
  
for (i = 0; i < N; i++)  
{  
pthread_mutex_destroy(&forks[i]);  
}  
return 0;  
}
```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Dining_Philosopher
Philosopher 0 is thinking.
Philosopher 1 is thinking.
Philosopher 2 is thinking.
Philosopher 3 is thinking.
Philosopher 4 is thinking.
Philosopher 0 picked LEFT fork 0
Philosopher 0 picked RIGHT fork 1
Philosopher 0 is eating.
Philosopher 1 picked RIGHT fork 2
Philosopher 4 picked LEFT fork 4
Philosopher 0 put down forks 0 and 1
Philosopher 0 is thinking.
Philosopher 4 picked RIGHT fork 0
Philosopher 4 is eating.
Philosopher 1 picked LEFT fork 1
Philosopher 1 is eating.
Philosopher 4 put down forks 4 and 0
Philosopher 4 is thinking.
Philosopher 1 put down forks 1 and 2
Philosopher 1 is thinking.
Philosopher 2 picked LEFT fork 2
Philosopher 2 picked RIGHT fork 3
Philosopher 2 is eating.
Philosopher 0 picked LEFT fork 0
Philosopher 2 put down forks 2 and 3
Philosopher 2 is thinking.
Philosopher 3 picked RIGHT fork 4
Philosopher 3 picked LEFT fork 3
Philosopher 3 picked LEFT fork 3
Philosopher 3 is eating.
Philosopher 1 picked RIGHT fork 2
Philosopher 1 picked LEFT fork 1
Philosopher 1 is eating.
Philosopher 0 picked LEFT fork 0
Philosopher 3 put down forks 3 and 4
Philosopher 3 is thinking.
Philosopher 0 picked RIGHT fork 1
Philosopher 0 is eating.
Philosopher 1 put down forks 1 and 2
Philosopher 1 is thinking.
Philosopher 2 picked LEFT fork 2
Philosopher 2 picked RIGHT fork 3
Philosopher 2 is eating.
Philosopher 3 picked RIGHT fork 4
Philosopher 3 put down forks 3 and 4
Philosopher 3 is thinking.
Philosopher 4 picked LEFT fork 4
Philosopher 1 picked RIGHT fork 2
Philosopher 1 picked LEFT fork 1
Philosopher 1 is eating.
Philosopher 0 picked LEFT fork 0
Philosopher 0 picked RIGHT fork 1
Philosopher 0 is eating.
```

Program – 7

Question :

Write a C program to simulate Bankers algorithm for the purpose of deadlock avoidance.

Code :

```
#include <stdio.h>

int main()
{
    int n, m, i, j, k;

    printf("Enter number of processes: ");
    scanf("%d", &n);

    printf("Enter number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], max[n][m], need[n][m];
    int avail[m], finish[n], safe[n];

    printf("Enter Allocation Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &alloc[i][j]);

    printf("Enter Maximum Demand Matrix:\n");
    for(i = 0; i < n; i++)
        for(j = 0; j < m; j++)
            scanf("%d", &max[i][j]);

    printf("Enter Available Resources:\n");
    for(i = 0; i < m; i++)
        scanf("%d", &avail[i]);

    for(i = 0; i < n; i++)
    {
        finish[i] = 0;
        for(j = 0; j < m; j++)
            need[i][j] = max[i][j] - alloc[i][j];
    }

    int count = 0;

    while(count < n)
    {
        int found = 0;

        for(i = 0; i < n; i++)
        {
```

```

if(finish[i] == 0)
{
    int flag = 1;

    for(j = 0; j < m; j++)
    {
        if(need[i][j] > avail[j])
        {
            flag = 0;
            break;
        }
    }

    if(flag)
    {
        for(k = 0; k < m; k++)
            avail[k] += alloc[i][k];

        safe[count++] = i;
        finish[i] = 1;
        found = 1;
    }
}

if(found == 0)
{
    printf("\nSystem is not in safe state\n");
    return 0;
}

printf("\nSystem is in a safe state.\n");
printf("Safe sequence is: ");

for(i = 0; i < n; i++)
{
    printf("P%d", safe[i]);

    if(i != n - 1)
        printf(" -> ");
}

printf("\n");

return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Bankers_algorithm
Enter number of processes: 5
Enter number of resources: 3
Enter Allocation Matrix:
0 1 0
2 0 0
3 0 2
2 1 1
0 0 2
Enter Maximum Demand Matrix:
7 5 3
3 2 2
9 0 2
2 2 2
4 3 3
Enter Available Resources:
3 3 2

System is in a safe state.
Safe sequence is: P1 -> P3 -> P4 -> P0 -> P2
```

⇒ Deadlock Detection

```
#include <stdio.h>
```

```
int main()
{
    int n, m, i, j, k;

    printf("Enter the number of processes: ");
    scanf("%d", &n);

    printf("Enter the number of resources: ");
    scanf("%d", &m);

    int alloc[n][m], request[n][m];
    int available[m], work[m];
    int finish[n], safe[n];

    printf("Enter the allocation matrix:\n");

    for(i = 0; i < n; i++)
    {
        printf("Process %d: ", i);
        for(j = 0; j < m; j++)
        {
            scanf("%d", &alloc[i][j]);
        }
    }
}
```

```

printf("Enter the request matrix:\n");

for(i = 0; i < n; i++)
{
    printf("Process %d: ", i);
    for(j = 0; j < m; j++)
    {
        scanf("%d", &request[i][j]);
    }
}

printf("Enter the available resources: ");

for(i = 0; i < m; i++)
{
    scanf("%d", &available[i]);
    work[i] = available[i];
}

for(i = 0; i < n; i++)
{
    finish[i] = 0;
}

int count = 0;

while(count < n)
{
    int found = 0;

    for(i = 0; i < n; i++)
    {
        if(finish[i] == 0)
        {
            int possible = 1;

            for(j = 0; j < m; j++)
            {
                if(request[i][j] > work[j])
                {
                    possible = 0;
                    break;
                }
            }

            if(possible)
            {
                for(k = 0; k < m; k++)
                {

```

```

        work[k] += alloc[i][k];
    }

    safe[count++] = i;
    finish[i] = 1;
    found = 1;
}
}
}

if(found == 0)
    break;
}

if(count == n)
{
    printf("System is in safe state.\n");
    printf("Safe Sequence is: ");

    for(i = 0; i < n; i++)
    {
        printf("P%d", safe[i]);

        if(i != n - 1)
            printf(" ");
    }

    printf("\n");
}
else
{
    printf("Deadlock detected.\n");
}

return 0;
}

```

Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Deadlock_Detection
Enter the number of processes: 5
Enter the number of resources: 3
Enter the allocation matrix:
Process 0: 0 1 0
Process 1: 2 0 0
Process 2: 3 0 3
Process 3: 2 1 1
Process 4: 0 0 2
Enter the request matrix:
Process 0: 0 0 0
Process 1: 2 0 2
Process 2: 0 0 0
Process 3: 1 0 0
Process 4: 0 0 2
Enter the available resources: 0 0 0
System is in safe state.
Safe Sequence is: P0 P2 P3 P4 P1
```


Program – 8

Question :

Write a C program to simulate the following contiguous memory allocation techniques.

- a) Worst-fit
- b) Best-fit
- c) First-fit

Code :

```
#include <stdio.h>

void firstFit(int blockSize[], int blocks, int processSize[], int processes)
{
    int allocation[processes];
    int used[blocks];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    for (int i = 0; i < blocks; i++)
        used[i] = 0;

    for (int i = 0; i < processes; i++)
    {
        for (int j = 0; j < blocks; j++)
        {
            if (!used[j] && blockSize[j] >= processSize[i])
            {
                allocation[i] = j;
                used[j] = 1;
                break;
            }
        }
    }

    printf("\n--- First Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < processes; i++)
    {
        printf("%d\t%d\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void bestFit(int blockSize[], int blocks, int processSize[], int processes)
```

```

{
    int allocation[processes];
    int used[blocks];

    for (int i = 0; i < processes; i++)
        allocation[i] = -1;

    for (int i = 0; i < blocks; i++)
        used[i] = 0;

    for (int i = 0; i < processes; i++)
    {
        int bestIdx = -1;

        for (int j = 0; j < blocks; j++)
        {
            if (!used[j] && blockSize[j] >= processSize[i])
            {
                if (bestIdx == -1 || blockSize[j] < blockSize[bestIdx])
                    bestIdx = j;
            }
        }

        if (bestIdx != -1)
        {
            allocation[i] = bestIdx;
            used[bestIdx] = 1;
        }
    }

    printf("\n--- Best Fit ---\n");
    printf("Process No.\tProcess Size\tBlock No.\n");

    for (int i = 0; i < processes; i++)
    {
        printf("%d\t\t%d\t\t", i + 1, processSize[i]);

        if (allocation[i] != -1)
            printf("%d\n", allocation[i] + 1);
        else
            printf("Not Allocated\n");
    }
}

void worstFit(int blockSize[], int blocks, int processSize[], int processes)
{
    int allocation[processes];
    int used[blocks];

```

```

for (int i = 0; i < processes; i++)
    allocation[i] = -1;

for (int i = 0; i < blocks; i++)
    used[i] = 0;

for (int i = 0; i < processes; i++)
{
    int worstIdx = -1;

    for (int j = 0; j < blocks; j++)
    {
        if (!used[j] && blockSize[j] >= processSize[i])
        {
            if (worstIdx == -1 || blockSize[j] > blockSize[worstIdx])
                worstIdx = j;
        }
    }

    if (worstIdx != -1)
    {
        allocation[i] = worstIdx;
        used[worstIdx] = 1;
    }
}

printf("\n--- Worst Fit ---\n");
printf("Process No.\tProcess Size\tBlock No.\n");

for (int i = 0; i < processes; i++)
{
    printf("%d\t\t%d\t\t", i + 1, processSize[i]);

    if (allocation[i] != -1)
        printf("%d\n", allocation[i] + 1);
    else
        printf("Not Allocated\n");
}
}

int main()
{
    int blocks, processes;

    printf("Enter number of memory blocks: ");
    scanf("%d", &blocks);

    int blockSize[blocks];

```

```

printf("Enter sizes of memory blocks:\n");

for (int i = 0; i < blocks; i++)
    scanf("%d", &blockSize[i]);

printf("Enter number of processes: ");
scanf("%d", &processes);

int processSize[processes];

printf("Enter sizes of processes:\n");

for (int i = 0; i < processes; i++)
    scanf("%d", &processSize[i]);

firstFit(blockSize, blocks, processSize, processes);
bestFit(blockSize, blocks, processSize, processes);
worstFit(blockSize, blocks, processSize, processes);

return 0;
}

```

Result :

```

C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>MemoryAllocation
Enter number of memory blocks: 5
Enter sizes of memory blocks:
100 500 200 300 600
Enter number of processes: 4
Enter sizes of processes:
212 417 112 426

--- First Fit ---
Process No.      Process Size      Block No.
1                212              2
2                417              5
3                112              3
4                426              Not Allocated

--- Best Fit ---
Process No.      Process Size      Block No.
1                212              4
2                417              2
3                112              3
4                426              5

--- Worst Fit ---
Process No.      Process Size      Block No.
1                212              5
2                417              2
3                112              4
4                426              Not Allocated

```

Program – 9

Question :

Write a C program to simulate page replacement algorithms.

a) FIFO

b) LRU

c) Optimal

Code :

```
#include <stdio.h>
#include <stdlib.h>

void fifo(int pages[], int n, int num_frames) {
    int *frames = (int *)malloc(num_frames * sizeof(int));
    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
    }
    int page_faults = 0;
    int next = 0;

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int present = 0;
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                present = 1;
                break;
            }
        }

        if (!present) {
            int space_available = 0;
            int empty_slot = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    space_available = 1;
                    empty_slot = j;
                    break;
                }
            }

            if (space_available) {
                frames[empty_slot] = page;
            } else {
                frames[next] = page;
                next = (next + 1) % num_frames;
            }
            page_faults++;
        }
    }
}
```

```

printf("FIFO Page Hits: %d\n", n - page_faults);
printf("FIFO Page Misses: %d\n", page_faults);
free(frames);
}

void lru(int pages[], int n, int num_frames) {
    int *frames = (int *)malloc(num_frames * sizeof(int));
    int *last_used = (int *)malloc(num_frames * sizeof(int));
    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
        last_used[i] = -1;
    }
    int page_faults = 0;

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int present = 0;
        int present_index = -1;
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                present = 1;
                present_index = j;
                break;
            }
        }

        if (present) {
            last_used[present_index] = i;
        } else {
            int space_available = 0;
            int empty_slot = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    space_available = 1;
                    empty_slot = j;
                    break;
                }
            }

            if (space_available) {
                frames[empty_slot] = page;
                last_used[empty_slot] = i;
            } else {
                int lru_index = 0;
                int min_time = last_used[0];
                for (int j = 1; j < num_frames; j++) {
                    if (last_used[j] < min_time) {
                        min_time = last_used[j];
                        lru_index = j;
                    }
                }
                frames[lru_index] = page;
                last_used[lru_index] = i;
            }
        }
    }
}

```

```

    }
    }
    frames[lru_index] = page;
    last_used[lru_index] = i;
}
page_faults++;
}
}
printf("LRU Page Hits: %d\n", n - page_faults);
printf("LRU Page Misses: %d\n", page_faults);
free(frames);
free(last_used);
}

```

```

void optimal(int pages[], int n, int num_frames) {
    int *frames = (int *)malloc(num_frames * sizeof(int));
    for (int i = 0; i < num_frames; i++) {
        frames[i] = -1;
    }
    int page_faults = 0;

    for (int i = 0; i < n; i++) {
        int page = pages[i];
        int present = 0;
        for (int j = 0; j < num_frames; j++) {
            if (frames[j] == page) {
                present = 1;
                break;
            }
        }

        if (!present) {
            int space_available = 0;
            int empty_slot = -1;
            for (int j = 0; j < num_frames; j++) {
                if (frames[j] == -1) {
                    space_available = 1;
                    empty_slot = j;
                    break;
                }
            }

            if (space_available) {
                frames[empty_slot] = page;
            } else {
                int replace_index = -1;
                int farthest = i;
                for (int j = 0; j < num_frames; j++) {
                    int next_use = -1;

```

```

        for (int k = i + 1; k < n; k++) {
            if (pages[k] == frames[j]) {
                next_use = k;
                break;
            }
        }
        if (next_use == -1) {
            replace_index = j;
            break;
        }
        if (next_use > farthest) {
            farthest = next_use;
            replace_index = j;
        }
    }
    frames[replace_index] = page;
}
page_faults++;
}
}
printf("Optimal Page Hits: %d\n", n - page_faults);
printf("Optimal Page Misses: %d\n", page_faults);
free(frames);
}

int main() {
    int n;
    if (scanf("%d", &n) != 1) return 1;

    int *pages = (int *)malloc(n * sizeof(int));
    for (int i = 0; i < n; i++) {
        if (scanf("%d", &pages[i]) != 1) return 1;
    }

    int num_frames;
    if (scanf("%d", &num_frames) != 1) return 1;

    fifo(pages, n, num_frames);
    lru(pages, n, num_frames);
    optimal(pages, n, num_frames);

    free(pages);
    return 0;
}

```


Result :

```
C:\Users\ADITYA\OneDrive\Desktop\ADITYA.P.M\1BM24CS021(OS LAB)>Page_Replacement
21
5 0 1 0 2 3 0 2 4 3 3 2 0 2 1 2 7 0 1 1 0
3
FIFO Page Hits: 10
FIFO Page Misses: 11
LRU Page Hits: 9
LRU Page Misses: 12
Optimal Page Hits: 12
Optimal Page Misses: 9
```